HR Assistant for Lumora Tech: Report

NLP course, Assignment 1: Build an NLP System. Repository: https://github.com/KOTAYE/nlp-assignment-1

The assistant answers employee questions **only from company policy, citing the exact section**. It **asks for missing information** first, handles **leave requests through a database reachable only via LLM tool calls**, and **routes unsupported questions to HR**. Every component is evaluated. The work followed **CRISP-DM**: the full phase log and iteration log are in `docs/crisp-dm.md`.

---

1. Business understanding: scope

Problem. HR answers the same questions again and again (leave, sick notes, benefits, expenses, documents) and processes leave requests by hand.

Users. - *Employees* ask policy questions and request, check and cancel leave. - *HR managers* review requests the rules could not decide, through the same chat.

Business logic kept in scope.

Capability	Rule
Policy Q&A	Answer only from the 9 Lumora Tech policies (98 sections), tailored to the employee's profile (full-time / part-time / contractor, office, probation), with a section citation in the reply text
Clarification	If the answer depends on a situational fact that is not known yet (illness duration, days abroad, expense type, document type), ask one question first
Leave requests	4 types. **Annual:** 24 days/yr, pro-rated in the first year, ≤ 5 carry-over days until 31 March, notice 14 days (> 3 days) or 2 working days, ≥ 2 teammates off → HR. **Sick:** ≤ 3 days without a certificate, 10 paid days/yr. **Unpaid:** ≤ 20 days, always HR. **Special:** 3 days per event. Contractors are not covered. Decisions are made by a deterministic rule engine: *approved / declined / pending_hr*
Tracking	Every request is stored with its status. Employees can list and cancel; HR can list pending requests and approve or decline them (with explicit confirmation)
DB access	The SQLite DB is read and written **only inside tools that the LLM decides to call**. The UI never queries it
HR referral	Sensitive topics (salary, harassment reports, disputes…) and questions the policies do not support become an HR ticket (category + summary) via a tool call

Out of scope: real authentication (a "log in as" selector stands in for SSO), payroll, notifications, document upload. **Success criteria:** see the targets in `crisp-dm.md` §1.6 and the results in §5.

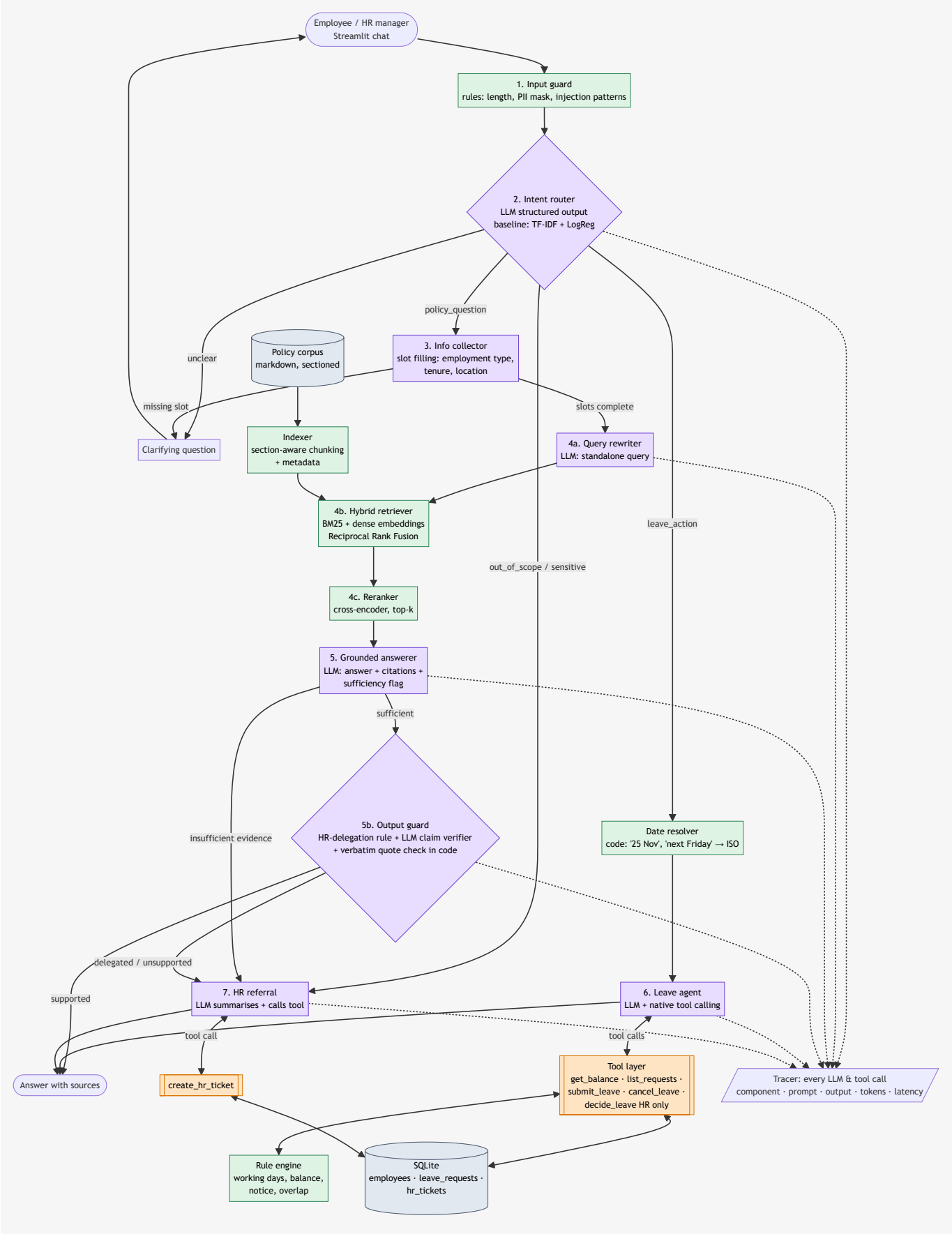
2. Data

- **Existing datasets were checked first** (`crisp-dm.md` §2):
- Hugging Face: syncora HR QA (MIT), several small instruction sets;
- GitHub: OpenGov HR manual (CC0);
- Kaggle and IEEE DataPort: nothing suitable or accessible.
- No open dataset combined a policy corpus, questions with gold sections, conditional rules and leave records. So:

- **Policy corpus:** synthetic, 9 documents, 98 numbered sections, ~4k words. The structure follows the OpenGov manual. It is designed for evaluation: conditional rules, cross-references, 4 tables, an outdated-rule trap, and explicit "handled by HR" sections.
- **Employee/leave seed:** 30 fictional employees, 14 historical requests. Each leave rule has at least one triggering case.
- **Eval data:** hand-labelled from the corpus. Realistic phrasings were adapted from syncora (labels re-assigned by hand). Datasets are split dev (~40%) / test (~60%); a **holdout** set was added after the first test run. Gold quality was checked automatically: every gold section exists, and every expected leave decision equals the rule engine's output.

Dataset	Rows	Used for
`qa`	62 (47 answerable + 15 must-refer)	retrieval, answerer, referral, oracle
`router`	78 (5 intents)	router + classical baseline
`collector`	44	information collection
`rewriter`	20 follow-ups	rewriter
`leave_agent`	26 scenarios	agent trajectory evaluation
`guard`	36 (18 attacks, 18 benign incl. PII)	input guard
`e2e`	30 multi-turn conversations	end-to-end
`judge_calibration`	19 controlled examples	judge reliability
`qa_holdout`, `agent_holdout`	18 + 7	unbiased estimate after iteration 2

3. System design



Legend. Purple = **LLM call** (OpenRouter, `openai/gpt-4o-mini`). Orange = **tool call** requested by the LLM, the only path to the DB. Green = local code or a local model. Grey = data.

#	Component	Purpose	Input → output	Model / tool calls
1	Input guard	Block prompt injection, mask PII	text → allowed + masked text	rules
2	Router	Choose the path	message + history → intent (5 classes)	LLM (JSON schema)
3	Info collector	Find which facts the answer needs and ask for missing ones	question + profile → facts / clarifying question	LLM + regex fallback
4a	Query rewriter	Make follow-ups standalone	history + question → query	LLM
4	Indexer	One citable chunk per `§`, with metadata, plus table-row units	markdown → chunks	local
4b	Hybrid retriever	Candidate sections	query → top-20	BM25 + `bge-small` dense, RRF
4c	Reranker	Precise ordering	query + 20 → top-5	cross-encoder (local)
5	Grounded answerer	Answer from evidence, cite, flag sufficiency	question + facts + top-5 → `{answer, citations, sufficient}`	LLM (JSON schema)
5b	Output guard	Stop answers the policy delegates to HR or that rely on the policy being silent	answer + cited sections → pass / route to HR	code rule + LLM verifier (asked item + quote) + code check that the quote is verbatim in the evidence
—	Date resolver	Natural date phrases → ISO dates for the agent	message → annotated message	code
6	Leave agent	Leave actions	message → tool calls → reply	LLM + tools
7	HR referral	Create a ticket	question → category + summary → ticket ID	LLM + tool
—	Rule engine	Leave decisions	request + DB state → status + reason + §	code
—	Tracer	Observability	every LLM/tool call → trace (UI + JSONL)	—

Walkthrough of one input: "*Do I need a sick note?*" (Andrii, full-time, Lviv). 1. The guard passes it. 2. The router returns `policy_question`. 3. The collector finds that the answer depends on `illness_duration_days`, which is unknown, so the assistant asks "*How many working days have you been ill?*". 4. The user replies "*4 days*". The pipeline resumes the stored question. 5. The rewriter produces "*sick leave 4 days medical certificate required*". 6. BM25 and dense rankings are fused with RRF into the top-20; the cross-encoder picks the top-5 with `sick-leave-policy §2.3` first. 7. The answerer replies "*More than 3 consecutive working days require a medical certificate; HR reviews the request [sick-leave-policy §2.3]*", with `sufficient=true`. 8. The trace shows 4 LLM calls with tokens, cost and latency.

For a leave message the router returns `leave_action`. The agent calls `submit_leave_request`, whose implementation runs the rule engine and writes the request with its status.

Why this is not vanilla RAG: routing between three kinds of sources (documents, DB, people), clarification before retrieval, hybrid retrieval with reranking validated by ablations, explicit abstention with an HR ticket, and a tool-using agent over a database with deterministic business rules and a human confirmation gate.

4. Process: CRISP-DM and iterations

Phase	What was done (details in `crisp-dm.md`)
Business understanding	Scope, leave rules, components, metric targets, all confirmed with the owner; compliance audit against the brief (traceability matrix)
Data understanding	Survey of open datasets; data strategy decided
Data preparation	Policy corpus, DB seed, gold datasets for every component with dev/test splits, automatic gold validation
Modeling	Rule engine (TDD, 41 tests); LLM wrapper with tracing; tools; retrieval stack; components; pipeline; UI
Evaluation	One eval script per component, calibrated judge, dev tuning → test → error analysis → iteration 2 → holdout
Deployment	Local Streamlit app, reset script, fixed business date for the demo, traces

The iteration log in `crisp-dm.md` records every change with its metric **before → after**. The main loops: 1. **Smoke tests:** - the agent mis-applied the New-Year rule; - the collector over-asked; - HR referred to list positions instead of request IDs. 2. **Dev tuning:** - guard patterns; - few-shot examples for the collector; - rewriter prompt (the first LLM rewrite *lost* to simple concatenation: MRR 0.619 vs 0.938); - answerer abstention rule; - referral category definitions. 3. **Test, iteration 1:** clear dev overfitting (see §5.3). 4. **Iteration 2,** with general fixes driven by first-failing-stage analysis: - table-row index units (Docling idea); - an answerer sufficiency rule in both directions; - a calendar for the agent; - narrower collector scope; - code-guaranteed sources in the reply. 5. **Holdout,** written *before* the iteration-2 fixes (see the git history), to keep an unbiased estimate. 6. **Iteration 3:** a new **holdout-2** focused on the remaining weak spots (answer vs refer, natural dates) was written and committed first, and the iteration-2 system was measured on it as a baseline. Only then were the planned fixes applied: - an output guard: a code rule for sections that delegate to HR plus an LLM claim verifier (the Ethics lecture's "hallucination guard"); - code-based date resolution for the agent; - a general router rule. 7. **Regression check:** iteration 3 introduced a date-context regression; it was caught on test, fixed with a unit test, and re-measured. 8. **Iteration 4: holdout-3** (topics the policies are silent about, privacy and "Can I …?" rule questions, sensitive cases) was written and measured first. Fixes: - verifier v2: the LLM must name the asked item and **quote** the evidence sentence about it, and **code checks that the quote really is in the evidence** (verbatim, whitespace/markdown-insensitive); - router v5: rule questions vs requests to act, and a narrower "personal sensitive case" class; - a **prompt/eval leakage check** (shared 6-word sequences between prompt files and eval items). 9. **Iteration 5: holdout-4** (answers in table rows, exceptions inside exclusion lists, "what should I do / whom do I contact" questions, named items the policies are silent about) was written and measured first. Its baseline exposed new error types (correctness 0.583). Fixes, preferring code over prompts: - *code:* citation recovery (the model sometimes garbles "§" in the citation list; IDs are now resolved by document + section number, also from inline references, and only IDs present in the evidence survive); - *code:* a quoted table row may skip columns, but every quoted cell must be in one and the same row; - *code:* a message with exactly one date and no range words is stated to the agent as a one-day request; - *code:* the collector no longer asks for the expense type when the question names the item; - *prompts:* router v6 (procedure questions vs personal incidents), rewriter v3 (query centred on the asked item, "reimbursed / not reimbursed"), verifier v3 (exceptions in exclusion lists, table rows). - A first version put the one-day rule into the agent prompt. It broke multi-day requests on test and holdout-2 (pass^3 1.0 → 0.933 / 0.833), so it was moved into code and both sets returned to 1.0.

5. Evaluation

Every component is evaluated (brief: "*evaluation is required for every component*"). Full auto-generated tables, including dev results, per-class reports and error lists: `docs/eval-results.md`. Raw per-row results: `evals/results/`.

5.1 Why these metrics

The same principle applies to every component: **what can be checked in code is checked in code**; an LLM judge is used only where meaning matters (AI Agents lecture: "assertions beat judges"). Every metric below answers a business question from §1.6.

Component	Metric(s)	Why this metric (and not another)	High vs low score means
Rule engine	unit-test pass rate over every rule in §1.4	The rules are deterministic, so a probabilistic metric would hide bugs. Any failure is a wrong leave decision.	100% is expected; anything less is a bug
Input guard	attack **recall** (block rate), **false-block rate**, PII masking accuracy	Errors cost differently: a missed attack is a security risk, while a false block annoys a real employee. A single accuracy would mix them (Evaluation lecture: precision/recall for asymmetric errors).	Low recall = attacks reach the LLM; high false-block rate = unusable
Indexer / chunker	section integrity (1 chunk per §, tables intact, metadata) + chunking ablation on Hit@5 / MRR	Citations need exactly one chunk per section ID. The ablation shows whether structure-aware chunking actually helps retrieval (RAG lecture: chunking strategies).	Integrity failures break citations; low MRR = evidence ranked too low
Router	**macro-F1**, **MCC**, per-class P/R, confusion matrix, **recall(hr_referral)**; vs TF-IDF+LogReg baseline	5 imbalanced classes, so macro-F1 weights rare classes equally and MCC is robust to imbalance (Evaluation lecture). Recall on hr_referral is the costly error (a sensitive case answered by a bot). The baseline shows what the LLM adds.	Low hr_referral recall = sensitive cases are not escalated
Info collector	situational-fact **P/R/F1**, value-extraction accuracy, clarification **P/R** ("asked iff needed")	The brief requires metrics for *information collection*. Fact recall = asking for what matters; precision = not over-asking. Clarification precision/recall separates "annoying" from "answering without the needed facts".	Low precision = needless questions; low recall = answers that ignore the user's situation
Query rewriter	key-term coverage + **retrieval ablation** (raw follow-up vs concat baseline vs LLM rewrite: Hit@5, MRR)	A rewrite has no single gold string. Its value is whether it makes retrieval work for follow-ups, so it is measured through the downstream metric (extrinsic) plus a cheap intrinsic check.	Lower MRR than the baseline = the rewriter adds noise
Retrieval	**Hit@k, Recall@k, MRR@10, nDCG@5** vs gold section IDs, for BM25 / dense / hybrid / +reranker	The brief requires *retrieval success*. Gold section IDs make it deterministic (no judge). Hit@5 = is the evidence available to the answerer; MRR/nDCG = is it ranked first, which matters because the context is short (IR lecture).	Hit@5 < 1 = the answerer cannot be right; low MRR = a weak reranking
Answerer	**correctness** (4-level judge vs gold answer, calibrated), **faithfulness** (supported claims / all claims, RAGAS definition), **citation hit rate**	The brief requires *answer correctness* and *source support*. Correctness needs semantics, so a judge is used, with BLEU/ROUGE rejected because paraphrases are valid (Evaluation lecture). Faithfulness catches hallucinations even when the answer happens to be right. Citation hit is deterministic.	Contradictions = wrong policy information (worst case); faithfulness < 1 = unsupported claims
Oracle answerer	the same metrics with gold context	Separates generation errors from retrieval errors ("measure retrieval and generation separately").	A gap between oracle and pipeline = retrieval loses evidence
HR referral	referral **precision / recall / F1** + ticket category accuracy	The brief requires *correct referral to HR*. Recall matters more: a confident wrong answer is worse than an extra ticket. Precision guards against "refer everything".	Low recall = hallucinated answers; low precision = useless assistant
Leave agent	trajectory layers: **outcome** (final DB state), **process** (required tools), **safety** (no forbidden tools / writes),	An agent can produce the right answer through a wrong process, or a fluent reply with a wrong DB state (AI Agents lecture: evaluate	Low pass^3 = unreliable in production even if pass@1 looks fine

Component	Metric(s)	Why this metric (and not another)	High vs low score means
	communication; efficiency (steps, tokens, cost); **pass^3**	the trajectory). pass^k measures consistency, because LLM runs differ even at temperature 0.	
Judge	agreement with **controlled examples** (exact level, pass/fail accuracy, contradiction recall)	"We need a benchmark for the judge" (Evaluation lecture). Correctness numbers are trustworthy only if the judge separates correct, partial and contradicting answers.	Low contradiction recall = the judge would hide wrong answers
End-to-end	scenario success rate (multi-turn, automatic checks), per-check pass rates, tracer coverage, cost and latency per conversation	Extrinsic evaluation: do the components work together in real conversations (clarification turns, follow-ups, HR approval flow)?	Component metrics can be high while hand-offs fail; this catches it

5.2 Setup

- **Splits and how each was used:**
- prompts and parameters were tuned on **dev**;
- **test** was used first to measure iteration 1;
- **holdout** was written before the iteration-2 fixes;
- **holdout-2** (boundary cases: answer vs refer, natural dates) was written before the iteration-3 fixes and first used to measure the *unchanged* iteration-2 system as a baseline;
- **holdout-3** and **holdout-4** were written the same way before iterations 4 and 5.
- The final system is therefore estimated without bias on **holdout-4**. Earlier sets were seen during development (their errors drove later fixes) and are reported for completeness.
- **Leakage check** (`python -m evals.check_leakage`): 43 eval items share a 6-word sequence with a prompt file, and 17 of them (20 item–prompt pairs) are fully contained in prompt examples. They are short generic phrasings ("Do I need a sick note?", "How many vacation days do I have left?") used as few-shot examples for the collector, rewriter and router, mostly from dev but some from test and one from holdout-3 (`h3a04`). Scores on those items are optimistic. **Holdout-4 has no overlap with any prompt.**
- **Business date** fixed at 2026-11-12. Every result file stores the model, judge, prompt versions, date and git commit. **All tables are generated from the results files** in `docs/eval-results.md` (`python -m evals.summarize`); the numbers below are copied from there.
- **Models:** generator `openai/gpt-4o-mini`; judge `google/gemini-2.5-flash`, a different family to avoid self-preference.
- **Cost:** the whole development, including all eval runs of five iterations, cost ≈ $1.5 (9,112 LLM calls, summed from the traces). One question costs ≈ $0.0006 and takes ≈ 5–6 s, which includes the output-guard call.

5.3 Results for the metrics required by the brief (final system, iteration 5)

Brief metric	How measured	Test	Holdout-3	**Holdout-4 (unbiased)**	Target (§1.6)
Information collection	collector situational-fact F1 / clarification accuracy	1.000 / 1.000	—	—	≥ 0.90 / ≥ 0.85
Retrieval success	Hit@5 / MRR@10 (hybrid + rerank); context recall@5 inside the pipeline	1.000 / 0.925; 0.929	1.000	**1.000**	≥ 0.90 / ≥ 0.75
Answer correctness	judge pass rate (fully or mostly right) over all answerable questions; contradiction rate	0.929; 0.000	1.000; 0.000	**1.000; 0.000**	≥ 0.85
Source support	faithfulness (supported claims / claims); citation hit rate	1.000; 1.000	0.940; 1.000	**1.000; 1.000**	≥ 0.90; ≥ 0.90
Correct referral to HR	precision / recall / F1 of "routed to HR"	0.818 / 1.000 / 0.900	1.000 / 0.900 / 0.947	**1.000 / 1.000 / 1.000**	≥ 0.85 / ≥ 0.95

All targets are met on holdout-4. Contradiction rate is 0.000 on every split: no remaining error is a confident wrong policy statement. Across all sets the weakest numbers are referral precision on test (0.818: two answerable questions referred because retrieval missed the section, which is the safe direction) and referral on the old holdout (P/R 0.750 / 0.750: one false and one missed referral out of 4 cases, see §5.5). Referral recall on holdout-2 / holdout-3 is 1.000 / 0.900. The runs differ slightly from run to run (§5.5, item 5).

5.4 Results per component

Component	Key result (test unless noted)	Reading
Rule engine	41/41 tests pass	The first run found a real bug (phantom carry-over from untracked 2025): 40 → 41
Input guard	attack block rate **0.70**, false-block rate **0.00**, PII masking **1.00** (dev after tuning: 1.00 / 0.00 / 1.00)	High precision, limited recall: the regexes generalise poorly to unseen phrasings (leetspeak "Ign0re", role-play "let's play a game", "repeat the text above"). These test misses were seen in an early combined run and deliberately **not** patched, so test stays honest
Indexer	98 sections → 98 chunks, tables intact, metadata present. Ablation (BM25): section-aware **MRR 0.850 / Hit@1 0.787** vs fixed 120-word windows **0.416 / 0.128**	Structure-aware chunking roughly doubles MRR. Fixed windows cut sections and mix their IDs
Router	LLM (v6): accuracy **0.957**, macro-F1 **0.961**, MCC **0.946**, recall(hr_referral) **0.778**. TF-IDF + LogReg baseline (trained on 32 dev rows): 0.391 / 0.275 / 0.201 / 0.222	The LLM wins by a wide margin with few labels. Since v5, general topics without a personal case (salary ranges, stock options, layoffs) are deliberately sent to `policy_question`, and the later stages refer them to HR. The router gold labels (written before this design change) still mark them `hr_referral`, which explains recall(hr_referral) 0.778 on test and 0.429 on dev. End to end these questions **are** referred: QA referral recall is 1.000 on dev and test
Info collector	facts P/R/F1 **1.0/1.0/1.0**, value extraction **1.0**, clarification P/R **1.0/1.0** (6 asks, 19 non-asks)	Dev went 0.72 → 1.0 F1 after few-shot examples. The test set is small (25 rows), so 1.0 means "no observed errors". Until iteration 5 the collector asked "what type of expense?" even when the item was named (5 pipeline errors across the holdouts). A code fallback removed all of them
Query rewriter	key-term coverage **1.0**. Retrieval on follow-ups (MRR@10): raw **0.413**, concat-previous-turn **0.819**, LLM rewrite **0.917**	v1 of the rewriter *lost* to concatenation on dev (0.619 vs 0.938) because it added noise. v2 restricted it to topic words and wins on test
Retrieval	Hit@5 / MRR@10: BM25 0.929 / 0.825 · dense 0.964 / 0.829 · hybrid RRF 0.964 / 0.876 · **+ reranker 1.000 / 0.925**	Each stage adds value: RRF fixes ranking where either retriever alone fails; the reranker puts the gold section first more often. Table-row units (iteration 2) make values hidden in tables retrievable
Answerer (oracle context)	answer rate with gold sections: iteration 1 **0.714** → iteration 2 **1.000**; correctness 1.000	Isolates generation: iteration 1 abstained too often even with perfect evidence
Output guard (iterations 3–5)	controlled set, verifier v4: unsupported claims caught **1.000**, precision **1.000**, false-rejection rate **0.000** (v1: 0.889 / 1.000 / 0.000; v2: 1.000 / 0.900 / 0.125)	v2 added the verbatim-quote check in code and caught every unsupported claim, but rejected a correct answer quoted from a table row. v3 accepts table rows cell by cell (still checked in code) and reads exceptions in exclusion lists; v4 also sees the facts collected in the conversation (after a clarification the question alone is vague). The 5 delegating sections are detected in code (unit test)
HR referral	P/R/F1: test 0.818/1.000/0.900, holdout 0.750/0.750/0.750, holdout-2 1.000/1.000/1.000, holdout-3 1.000/0.900/0.947, **holdout-4 1.000/1.000/1.000**; ticket category accuracy 1.000 / 1.000 / 0.900 / 0.778 / 1.000	See §5.5

Component	Key result (test unless noted)	Reading
Date resolver (iterations 3, 5)	unit tests on natural phrases; effect: agent pass^3 on natural-date scenarios (holdout-2) **0.333** → **1.000**; one-day requests (holdout-4) **0.667** → **1.000**	LLMs mis-compute weekdays and years, and asked for an end date on one-day requests; code resolves both
Leave agent	pass^3 **1.000** on test and all four holdouts (outcome, process, **safety** and communication all 1.000); dev pass@1 0.909; 2.1 steps and $0.0004 per task on test	No forbidden tool was ever called, and no permission or confirmation was bypassed. The dev miss (ag23): the agent sent a wrong year, received the tool's instructive error and asked the user instead of retrying
Judge	controlled examples: exact level **0.947**, pass/fail accuracy **1.000**, contradiction recall **1.000**	The judge can be trusted to separate correct, partial and wrong answers
End-to-end	test success **0.944** (17/18), dev **1.000**, tracer coverage **1.000**, ≈ $0.0007 per conversation	The one failure is the "home internet" case in §5.5; it passed in the first iteration-5 run (1.000), so it is at the run-to-run noise level. A 0.889 run in between exposed a real bug (`e2e07` : after a clarification the verifier saw only the vague first question), which verifier v4 fixed

5.5 Why the scores are not higher, and what would fix them

Errors are classified by the **first stage that loses the answer** (RAG lecture). All remaining errors of the final system across dev, test and the four holdouts (99 QA questions, 30 E2E scenarios):

1. **Silence-based answers that apply a neighbouring rule** (holdout `hu01` "bigger monitor", E2E `e2e25` "home internet"). The answerer replies with a true rule about a *different* item ("office-based employees are not eligible for the home office allowance"), and the verifier quotes that sentence as support. This is the costliest remaining error type, although no false policy statement is made. - *Tried and rejected:* a code check that the quote must share a content word with the asked item. Offline on 117 logged verifier decisions it would have rejected 29, almost all correct (hotel ↔ accommodation, laptop ↔ equipment, bullied ↔ harassment). So it was **not** shipped. - *Next:* a second, narrow LLM check ("does this sentence talk about X? yes/no"), or self-consistency over 3 verifier runs. Both should be measured on the same sets.
2. **Retrieval misses → safe false referral** (test `qa12` , `qa14` : "Can I request leave through the HR assistant?", "Can I cancel a vacation I booked?"). The gold section is not in the top 5, so the system refers to HR instead of answering. *Next:* query expansion with the original question (multi-query + RRF).
3. **Over-cautious answerer** (holdout `hq14` , "Is the office open after 9 pm?"). The evidence says 08:00–21:00, but the answerer judged it insufficient. This happens only in some runs.
4. **Ambiguous gold label** (holdout-3 `h3u08` , "How does paternity leave work here?"). `leave-policy §4.1` grants 3 days for the birth of a child, while §7.1 sends parental leave to HR. The system answers with §4.1, and the gold label expects a referral. Both readings are defensible.
5. **Run-to-run variance.** The same code gave E2E test 1.000 and 0.944, and holdout referral precision 1.000 and 0.750, in different runs. With 4–10 referral cases per set, one decision moves a metric by 0.1–0.25. Single numbers are point estimates. The consistent direction across five independent sets is the stronger evidence.
6. **Guard recall 0.70.** Rules cannot cover paraphrased attacks. *Next:* a small learned injection classifier. Its impact is already limited by defence in depth: HR tools are not offered to employees, decisions need confirmation, and DB writes go only through validated tools.

5.6 Iteration comparison

Generated by `evals/summarize.py` from the results snapshots (`evals/results_iteration{1..4}/` and `evals/results/`).

Test split across iterations (seen during development, so optimistic from iteration 2 on):

metric	iter 1	iter 2	iter 3	iter 4	iter 5
Answer correctness (pass rate)	0.786	0.857	0.893	0.857	**0.929**
Faithfulness	0.986	1.000	1.000	1.000	**1.000**
Citation hit rate	0.917	1.000	1.000	1.000	**1.000**
HR referral precision	0.700	0.667	0.800	0.692	**0.818**
HR referral recall	0.778	0.667	0.889	1.000	**1.000**
Oracle answer rate	0.714	1.000	1.000	1.000	**1.000**
Leave agent pass^3	0.800	0.933	1.000	1.000	**1.000**
E2E scenario success	0.722	0.833	0.889	0.944	**0.944**

Fresh holdout sets, each measured before → after the iteration it was written for (the unbiased evidence):

metric	holdout-2: iter 2 → 3	holdout-3: iter 3 → 4	holdout-4: iter 4 → 5
Answer correctness (pass rate)	0.833 → 0.917	0.929 → 1.000	**0.583 → 1.000**
Faithfulness	0.975 → 1.000	0.936 → 0.940	**0.929 → 1.000**
HR referral precision	0.818 → 0.900	1.000 → 1.000	**0.600 → 1.000**
HR referral recall	0.900 → 0.900	0.700 → 0.800	**1.000 → 1.000**
Leave agent pass^3	0.333 → 1.000	1.000 → 1.000	**0.667 → 1.000**

Reading the tables: - **Iteration 1 → 2.** Test showed dev overfitting. Fixing over-abstention (oracle answer rate 0.714 → 1.0) and table retrieval raised correctness, but referral recall fell: the answerer now answered some cases it should refer. - **Iteration 2 → 3.** The output guard (code delegation rule + LLM claim verifier) restored referral quality while keeping the gains. The date resolver fixed the agent's calendar errors. - **Iteration 3 → 4.** The verbatim-quote check pushed referral recall on test to 1.000, but over-referred (precision 0.692): table-row quotes failed the strict check, and garbled citations dropped valid answers. - **Iteration 4 → 5.** Holdout-4 was built around exactly these error types and the baseline confirmed them (correctness 0.583, precision 0.600). The fixes, mostly in code, brought every holdout-4 metric to 1.000 and recovered test precision (0.692 → 0.818) without losing recall. - **Regressions caught and fixed.** Iteration 3: a bare "the 14th" was resolved to the wrong month (test caught it, unit test added). Iteration 5: a prompt rule for one-day leave broke multi-day requests (test and holdout-2 caught it, so the rule moved into code); and the verifier rejected a correct answer after a clarification (E2E caught it, so v4 passes the collected facts). Each fix shipped with a unit test or an eval re-run.

6. AI coding assistant: how it was managed

- `CLAUDE.md` **(agent rules file):**
- the **assignment brief has the highest priority**, with a traceability matrix (`docs/requirements-traceability.md`) updated before closing any phase, and the hard constraints listed explicitly;

- **CRISP-DM is mandatory**: every task names its phase and ends with a phase-log and iteration-log update;
- code conventions: one LLM wrapper with tracing, versioned prompt files, Pydantic structured outputs, DB only through tools, configuration only from `.env`;
- evaluation rules: "assertions beat judges", dev/test split, first-failing-stage analysis, *never write a number that is not in a results file*;
- git rules.
- **Skills** (`.claude/skills/`):
- `crisp-dm-step` (phase entry and exit);
- `add-component` (definition-of-done checklist: schema, prompt, tracing, tests, eval, baseline, docs);
- `build-eval-dataset` (coverage of hard cases, splits, data card);
- `run-evals` (metrics, calibration, interpretation).
- **Context management:**
- the course lectures were condensed once into `docs/course-notes.md` and never loaded whole;
- datasets were sampled, not dumped;
- results are summarised by `evals/summarize.py`.
- **Quality gates:** `ruff` (lint + format), `pytest` (63 tests), eval results stored with prompt versions, model and git commit.
- **What the rules caught or prevented:**
- a component writing to the DB without the LLM (fixed after the compliance audit);
- a test-set example about to leak into a prompt;
- a wrong gold label (#9 vs #10);
- optimistic dev numbers, which were reported as such, with a holdout added.

7. Limitations and next steps

- Silence-based answers that apply a neighbouring rule still pass the verifier in some runs ("bigger monitor", "home internet"). A lexical check was tried and rejected because it made too many false rejections. Next: a narrow second check or self-consistency (§5.5).
- The guard is rule-based. It does not generalise to new attack phrasings, so a learned classifier is the next step. Defence in depth already limits the impact: tools enforce permissions, and HR decisions need confirmation.
- The eval sets are small (tens of rows per component; 4–10 referral cases per set), so one row moves a metric by 0.1 or more. The next step is more data, including real employee phrasing.
- Some few-shot examples overlap with dev and test items (§5.2). Holdout-4 has no overlap and is the reference estimate.
- The synthetic corpus is cleaner than real handbooks (no scanned PDFs, no conflicting versions across documents).
- LLM non-determinism exists even at temperature 0. It is measured with pass^k and visible as run-to-run differences of one scenario. Production would need retries or self-consistency for critical paths.